



New Mansoura University  
Faculty of Computer Science and Engineering



# Advanced Machine Learning

## Gaussian Mixture Model

Islam Reda

Associate Professor

[islam-cis@mans.edu.eg](mailto:islam-cis@mans.edu.eg)



# Outline

- Gaussian Mixtures
  - Anomaly Detection using Gaussian Mixtures
  - Selecting the Number of Clusters
  - Bayesian Gaussian Mixture Models



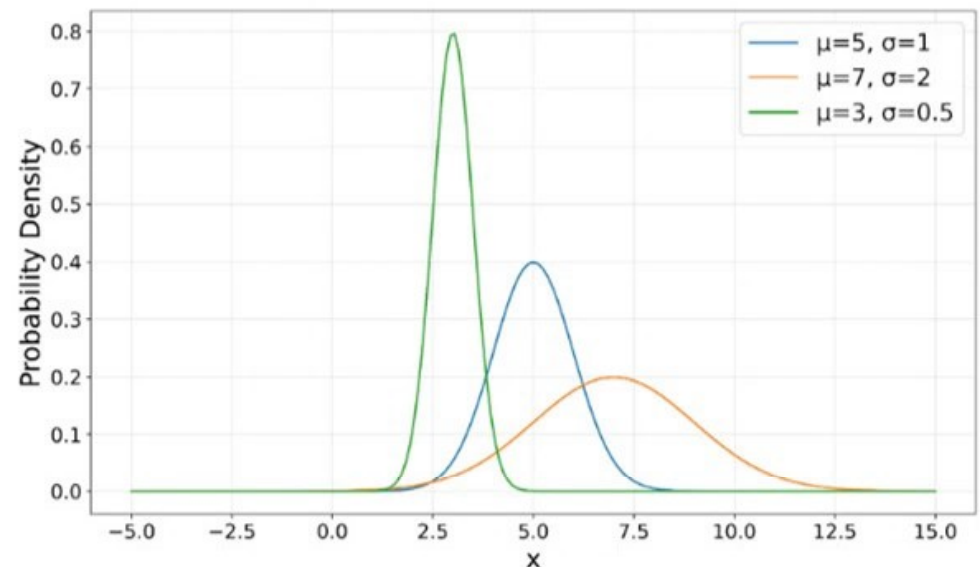
# Gaussian Mixture Model

# Gaussian Distribution

- A Gaussian distribution in one variable has a probability density function (PDF):

$$p(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-(x-\mu)^2/2\sigma^2}$$

Controlled by  $\mu$ (center) and  $\sigma$ (spread)

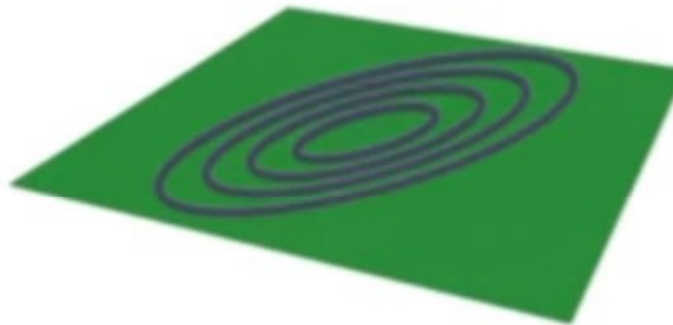
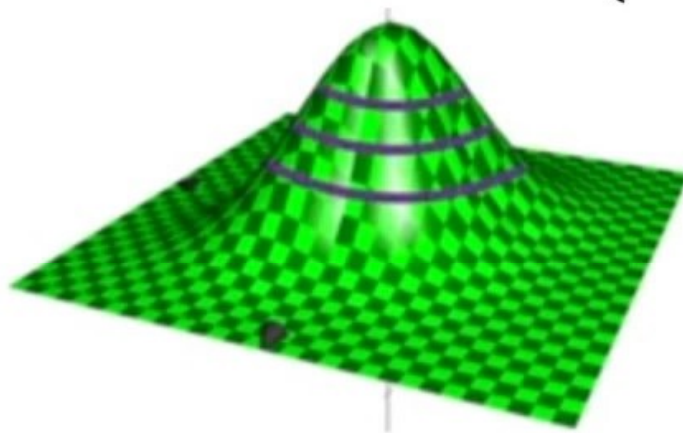


Various Gaussian distributions

# Multivariate Gaussian Distribution

- A multivariate Gaussian distribution has a PDF:

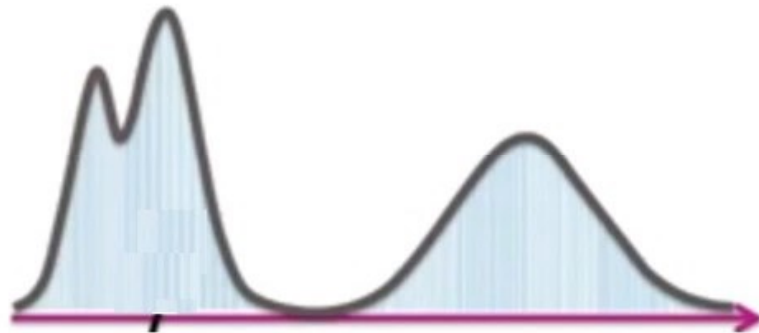
$$p(x; \mu, \Sigma) = \frac{1}{(2\pi)^{\frac{n}{2}} |\Sigma|^{\frac{1}{2}}} e^{-\frac{1}{2}(x-\mu)^T \Sigma^{-1} (x-\mu)}$$



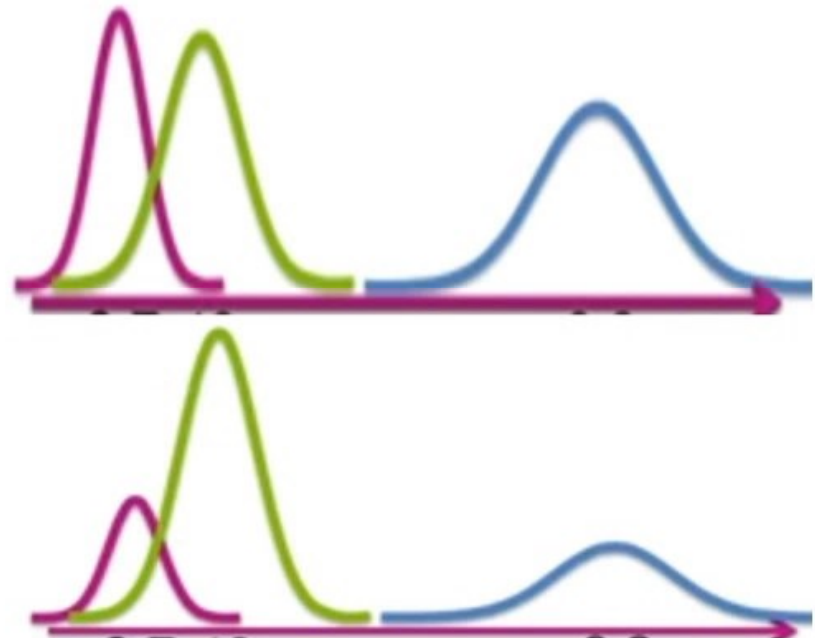
A 3D Gaussian distribution and its representation on 2D space using contour lines

# Gaussian Mixtures

- A **Gaussian mixture model (GMM)** is a probabilistic model that assumes that the instances were generated from a mixture of several Gaussian distributions whose parameters are unknown.



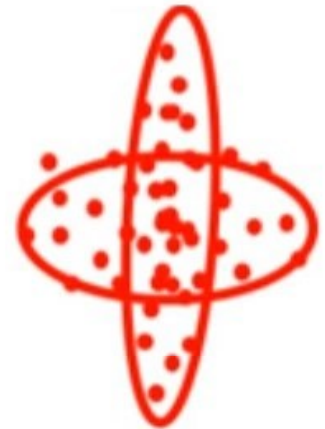
A histogram of an image





# Gaussian Mixtures

- K-means assigns each data point to a single cluster (hard clustering).
- What if clusters are overlapping?
  - Hard to tell which cluster is right (try to remain uncertain).
- GMM
  - Clusters are modeled as Gaussians.
  - Expectation-Maximization algorithm: assigns data to cluster with some probability.





# Gaussian Mixtures

- All the instances generated from a single Gaussian distribution form a cluster that typically looks like an ellipsoid.
- Each cluster can have a different ellipsoidal shape, size, density and orientation.
- When you observe an instance, you know it was generated from one of the Gaussian distributions, but you are not told which one, and you do not know what the parameters of these distributions are.
- There are several GMM variants: in the simplest variant, implemented in the **GaussianMixture** class, you must know in advance the number  $k$  of Gaussian distributions.



# Gaussian Mixtures

- Given the dataset  $\mathbf{X}$ , you typically want to start by estimating the weights  $\boldsymbol{\phi}$  and all the distribution parameters  $\boldsymbol{\mu}^{(1)}$  to  $\boldsymbol{\mu}^{(k)}$  and  $\boldsymbol{\Sigma}^{(1)}$  to  $\boldsymbol{\Sigma}^{(k)}$ . Scikit-Learn's GaussianMixture class makes this trivial:

```
from sklearn.mixture import GaussianMixture
```

```
gm = GaussianMixture(n_components=3, n_init=10)  
gm.fit(X)
```

- Let's look at the parameters that the algorithm estimated:

```
>>> gm.weights_  
array([0.20965228, 0.4000662 , 0.39028152])
```



# Gaussian Mixtures

```
>>> gm.means_  
array([[ 3.39909717,  1.05933727],  
       [-1.40763984,  1.42710194],  
       [ 0.05135313,  0.07524095]])  
>>> gm.covariances_  
array([[[[ 1.14807234, -0.03270354],  
          [-0.03270354,  0.95496237]],  
  
        [[ 0.63478101,  0.72969804],  
         [ 0.72969804,  1.1609872 ]],  
  
        [[0.68809572,  0.79608475],  
         [0.79608475,  1.21234145 ]]])]
```



# Gaussian Mixtures

- GaussianMixture class relies on the **Expectation-Maximization (EM)** algorithm, which has many similarities with the K-Means algorithm:
- It also initializes the cluster parameters randomly, then it repeats two steps until convergence,
  - Assigning instances to clusters (this is called the **expectation step**).
  - Updating the clusters (this is called the **maximization step**).
- EM can be considered as a generalization of K-Means which not only finds the cluster centers ( $\mu^{(1)}$  to  $\mu^{(k)}$ ), but also their size, shape and orientation ( $\Sigma^{(1)}$  to  $\Sigma^{(k)}$ ), as well as their relative weights ( $\phi^{(1)}$  to  $\phi^{(k)}$ ).



# Gaussian Mixtures

- Unlike K-Means, **EM uses soft cluster assignments** rather than hard assignments.
- For each instance during the expectation step, the algorithm estimates the probability that it belongs to each cluster (based on the current cluster parameters).
- During the maximization step, each cluster is updated using all the instances in the dataset, with each instance weighted by the estimated probability that it belongs to that cluster.
- These probabilities are called the **responsibilities** of the clusters for the instances.



# Gaussian Mixtures

- Unfortunately, just like K-Means, **EM can end up converging to poor solutions**, so it needs to be run several times, keeping only the best solution. This is why we set `n_init` to 10. Be careful: **by default, `n_init` is only set to 1**.
- To check whether or not the algorithm converged and how many iterations it took:

```
>>> gm.converged_  
True  
>>> gm.n_iter_  
3
```



# Gaussian Mixtures

- The model can easily assign each instance to the most likely cluster (hard clustering) or estimate the probability that it belongs to a particular cluster (soft clustering).
- Use the **predict()** method for hard clustering, or the **predict\_proba()** method for soft clustering.

```
>>> gm.predict(X)
array([2, 2, 1, ..., 0, 0, 0])
>>> gm.predict_proba(X)
array([[2.32389467e-02, 6.77397850e-07, 9.76760376e-01],
       [1.64685609e-02, 6.75361303e-04, 9.82856078e-01],
       [2.01535333e-06, 9.99923053e-01, 7.49319577e-05],
       ...,
       [9.99999571e-01, 2.13946075e-26, 4.28788333e-07],
       [1.00000000e+00, 1.46454409e-41, 5.12459171e-16],
       [1.00000000e+00, 8.02006365e-41, 2.27626238e-15]])
```



# Gaussian Mixtures

- It is also possible **to estimate the density of the model at any given location**. This is achieved using the **score\_samples()** method: for each instance it is given, this method estimates the log of the probability density function (PDF) at that location.
- The greater the score, the higher the density:

```
>>> gm.score_samples(X)  
array([-2.60782346, -3.57106041, -3.33003479, ..., -3.51352783,  
       -4.39802535, -3.80743859])
```



# Gaussian Mixtures

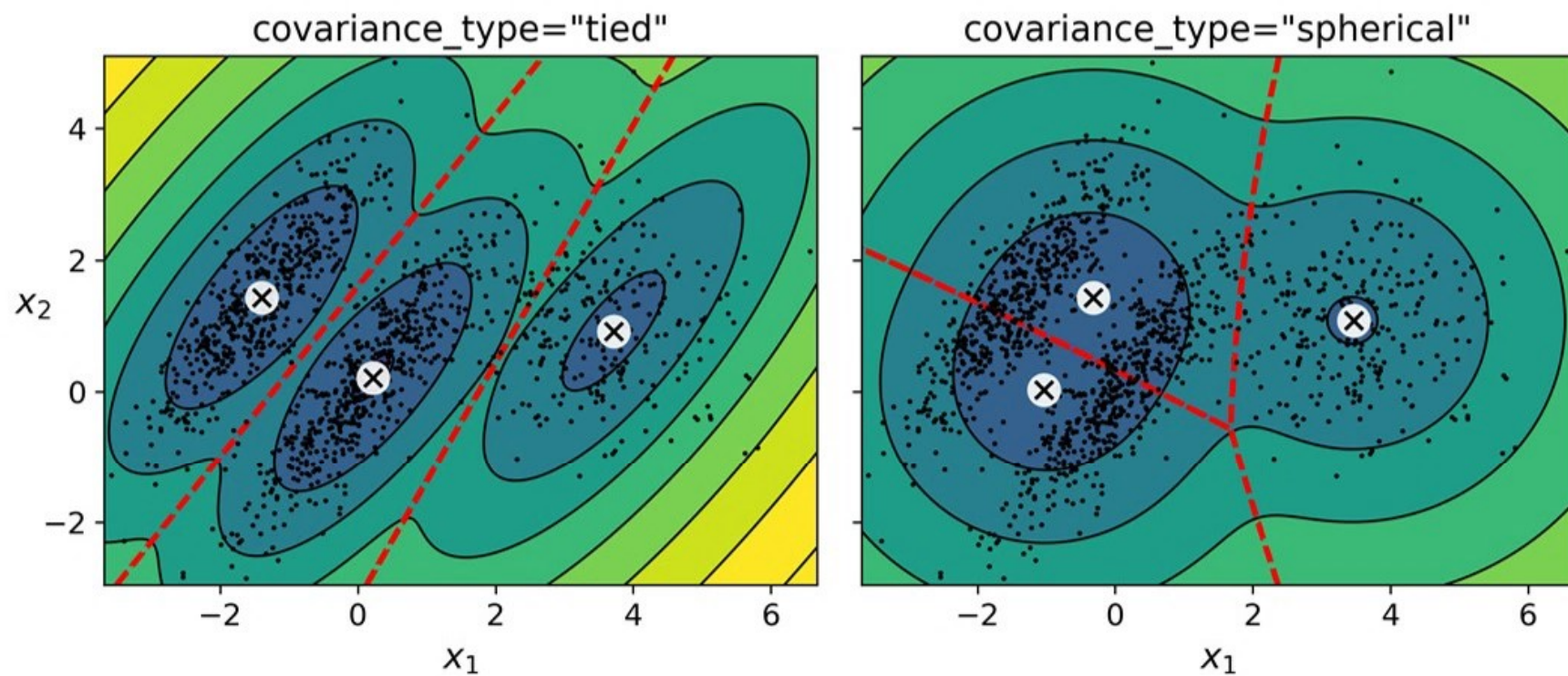
- When there are many dimensions, or many clusters, or few instances, EM can struggle to converge to the optimal solution.
- You might need to **reduce the difficulty of the task** by limiting the number of parameters that the algorithm has to learn:
- one way to do this is to limit the range of shapes and orientations that the clusters can have. This can be achieved by imposing constraints on the covariance matrices. To do this, just set the **covariance\_type** hyperparameter to one of the following values:



# Gaussian Mixtures

- "**spherical**": all clusters must be spherical, but they can have different diameters (i.e., different variances).
- "**diag**": clusters can take on any ellipsoidal shape of any size, but the ellipsoid's axes must be parallel to the coordinate axes (i.e., the covariance matrices must be diagonal).
- "**tied**": all clusters must have the same ellipsoidal shape, size and orientation (i.e., all clusters share the same covariance matrix).
- By default, `covariance_type` is equal to "**full**", which means that each cluster can take on any shape, size and orientation (it has its own unconstrained covariance matrix).

# Gaussian Mixtures



covariance\_type diagram



# Anomaly Detection using Gaussian Mixtures



# Anomaly Detection using Gaussian Mixtures

- **Anomaly detection** (also called **outlier detection**) is the task of detecting instances that deviate strongly from the norm.
- These instances are of course called **anomalies** or **outliers**, while the **normal instances** are called **inliers**.
- Anomaly detection is very useful in a wide variety of applications:
  - Detecting defective products in manufacturing.
  - Removing outliers from a dataset before training another model, which can significantly improve the performance of the resulting model.
  - Fraud detection.

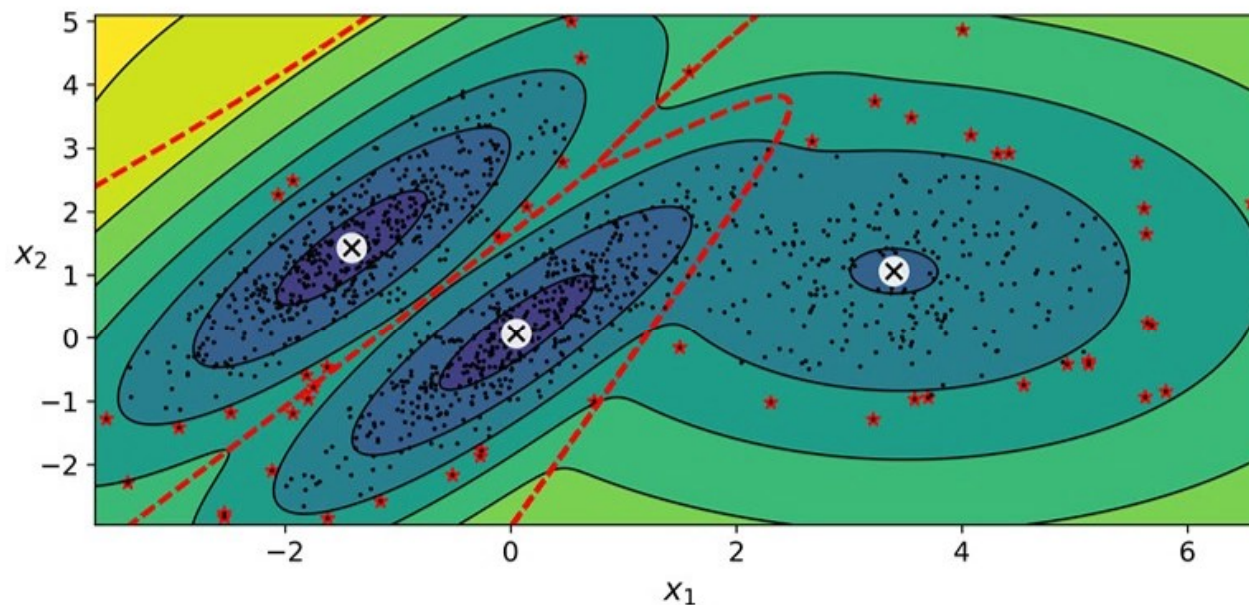


# Anomaly Detection using Gaussian Mixtures

- Using a Gaussian mixture model for anomaly detection is simple:
  - Any instance located in a low-density region can be considered an anomaly.
- You must define what density threshold you want to use.
- For example, in a manufacturing company that tries to detect defective products, the ratio of defective products is usually well-known (Say it is equal to 4%).
- If you notice that you get too many false positives (i.e., perfectly good products that are flagged as defective), you can lower the threshold.
- Conversely, if you have too many false negatives (i.e., defective products that the system does not flag as defective), you can increase the threshold.

# Anomaly Detection using Gaussian Mixtures

```
densities = gm.score_samples(X)
density_threshold = np.percentile(densities, 4)
anomalies = X[densities < density_threshold]
```



Anomaly detection using a Gaussian mixture model



# Selecting the Number of Clusters

- Just like K-Means, the GaussianMixture algorithm requires you to specify the number of clusters. So how can you find it?
- With K-Means, you could use the inertia or the silhouette score to select the appropriate number of clusters.
- But with Gaussian mixtures, it is not possible to use these metrics because they are not reliable when the clusters are not spherical or have different sizes.
- Instead, you can try to find the model that minimizes a theoretical information criterion such as the Bayesian information criterion (BIC) or the Akaike information criterion (AIC).



# Selecting the Number of Clusters

$$BIC = \log(m)p - 2 \log(\hat{L})$$

$$AIC = 2p - 2 \log(\hat{L})$$

$m$  is the number of instances, as always.

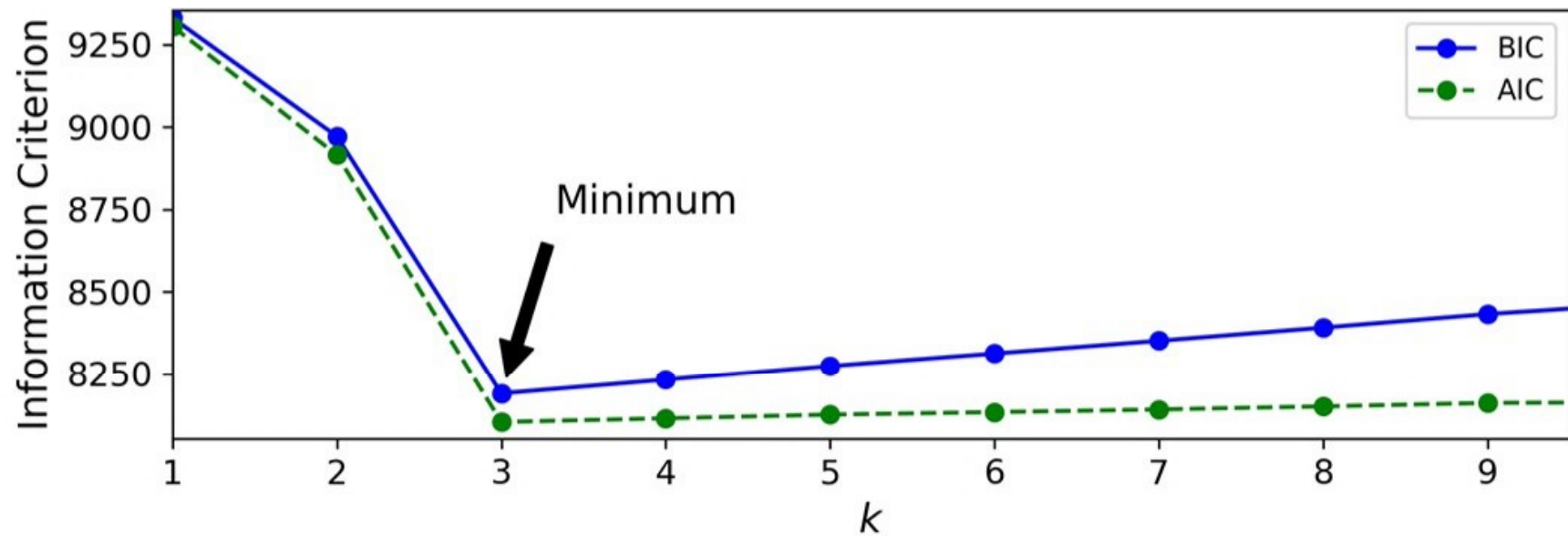
$p$  is the number of parameters learned by the model.

$\hat{L}$  is the maximized value of the likelihood function of the model.

- Both the BIC and the AIC penalize models that have more parameters to learn (more clusters), and reward models that fit the data well.
- To compute the BIC and AIC, just call the `bic()` or `aic()` methods:

```
>>> gm.bic(X)
8189.74345832983
>>> gm.aic(X)
8102.518178214792
```

# Selecting the Number of Clusters



AIC and BIC for different numbers of clusters  $k$



# Bayesian Gaussian Mixture Models

- Rather than manually searching for the optimal number of clusters, it is possible to use instead the **BayesianGaussianMixture** class which is capable of giving **weights equal (or close) to zero to unnecessary clusters**.
- Just set the number of clusters **n\_components** to a value that you have good reason to believe is greater than the optimal number of clusters, and the algorithm will eliminate the unnecessary clusters automatically.

```
>>> from sklearn.mixture import BayesianGaussianMixture
>>> bgm = BayesianGaussianMixture(n_components=10, n_init=10, random_state=42)
>>> bgm.fit(X)
>>> np.round(bgm.weights_, 2)
array([0.4 , 0.21, 0.4 , 0. , 0. , 0. , 0. , 0. , 0. , 0. ])
```



**Thank You**